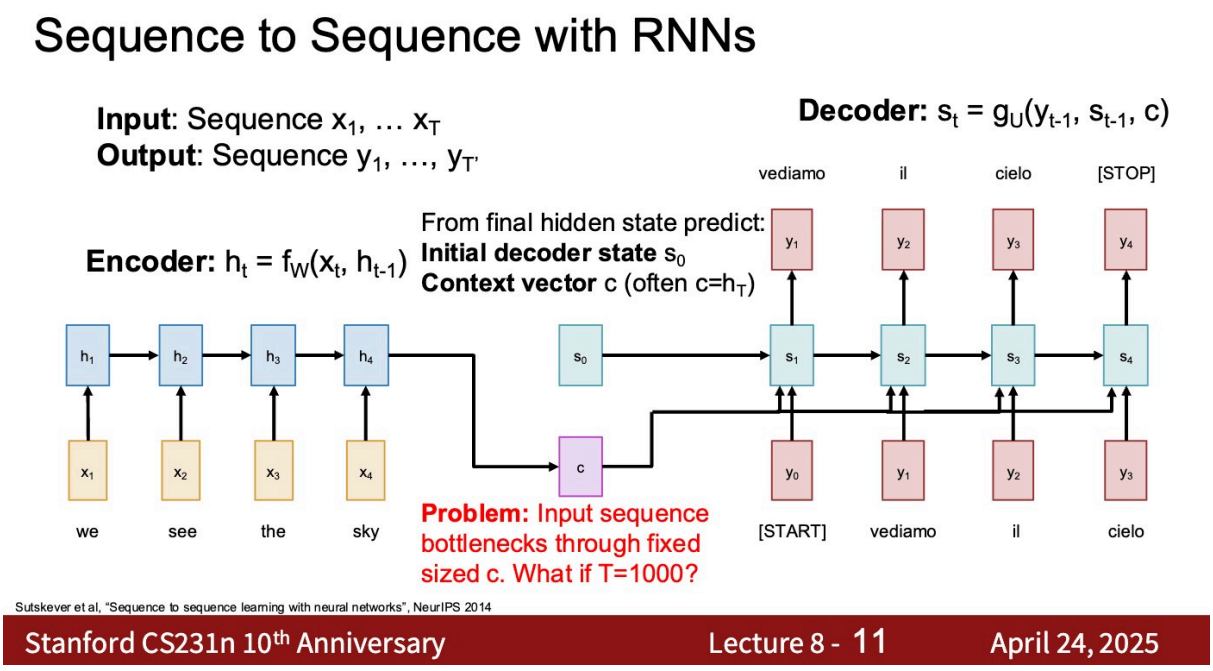
Lecture 8-9

Attention

: 디코더가 단어를 만들 때마다 입력 전체를 다시 보고 필요한 곳에 집중한다.

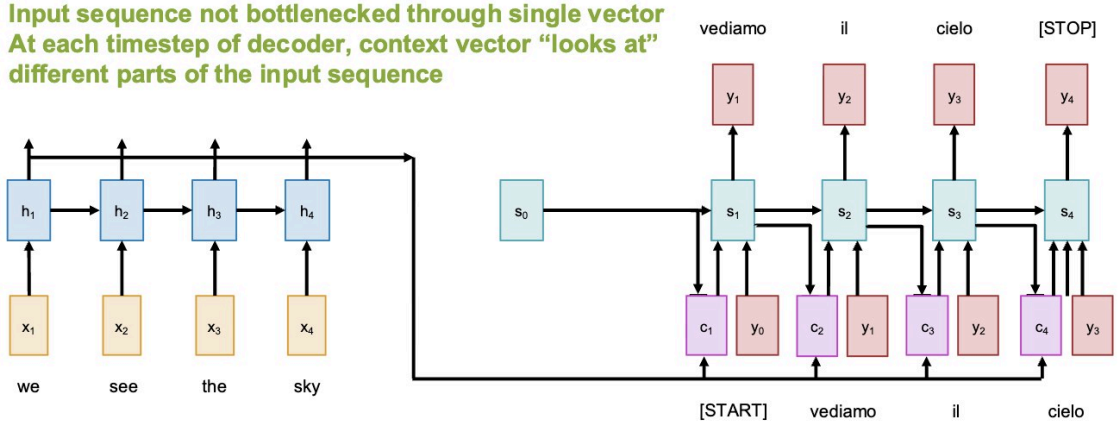


- 기존 seq2seq : 입력 전체를 벡터 c 하나에 담는다 → 문장이 길면 정보 손실 (병목)
- Attention : 단계마다 입력 h들과의 유사도 → softmax 가중치 → 가중합으로 새 context 벡터 c_t를 만든다.
- 어디를 볼지는 정답 없이 역전파로 스스로 배운다.

Sequence to Sequence with RNNs and Attention

Use a different context vector in each timestep of decoder

- **Input sequence not bottlenecked through single vector**
- **At each timestep of decoder, context vector "looks at" different parts of the input sequence**



Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

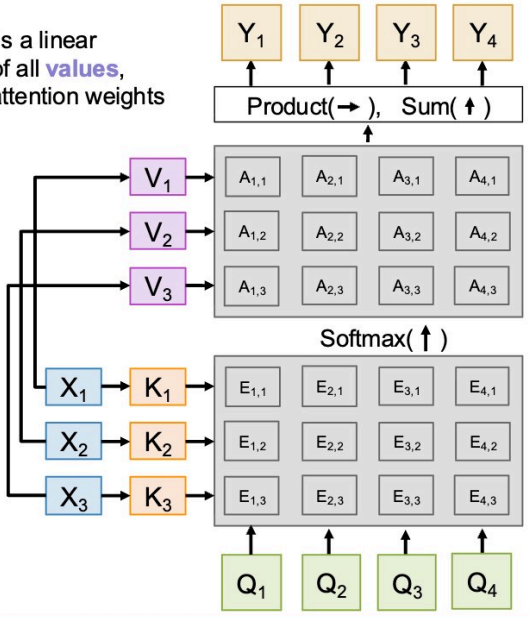
Attention Layer

: 쿼리가 키와 비교해 가중치를 구하고 그 가중치로 값을 섞는다.

- Query : 찾는 것 / Key : 비교 대상 / Value : 실제로 섞는 내용
- 유사도 = QK^T / $\sqrt{D}$. $\sqrt{D}$로 나눠 softmax가 한쪽으로 쏠려 기울기가 사라지는 것을 막는다.
- Self-Attention : Q, K, V를 모두 같은 입력에서 만든다. / Cross-Attention : Q와 K, V가 다른 입력에서 온다.

Attention Layer

Each **output** is a linear combination of all **values**, weighted by attention weights



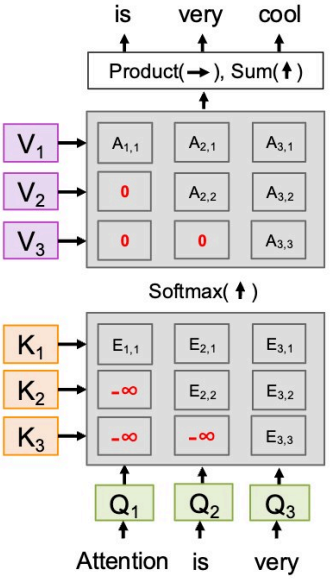
Inputs:
Query vector: Q [N_Q x D_Q]
Data vectors: X [N_X x D_X]
Key matrix: W_K [D_X x D_Q]
Value matrix: W_V [D_X x D_V]

Computation:
Keys: $K = XW_K$ [N_X x D_Q]
Values: $V = XW_V$ [N_X x D_V]
Similarities: $E = QK^T / \sqrt{D_Q}$ [N_Q x N_X]
$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$
Attention weights: A = softmax(E, dim=1) [N_Q x N_X]
Output vector: $Y = AV$ [N_Q x D_V]
$Y_i = \sum_j A_{ij} V_j$

- 순서를 모른다 ⇒ 위치 인코딩을 더한다. →
- Masked : 미래 단어 점수를 -∞로 → 앞 단어만 본다. (언어 모델)
- Multi-head : 여러 헤드가 서로 다른 관계를 보고 결과를 이어 붙인다.
- (-) 계산량과 메모리가 길이 N^2에 비례

Masked Self-Attention Layer

Don't let vectors "look ahead" in the sequence



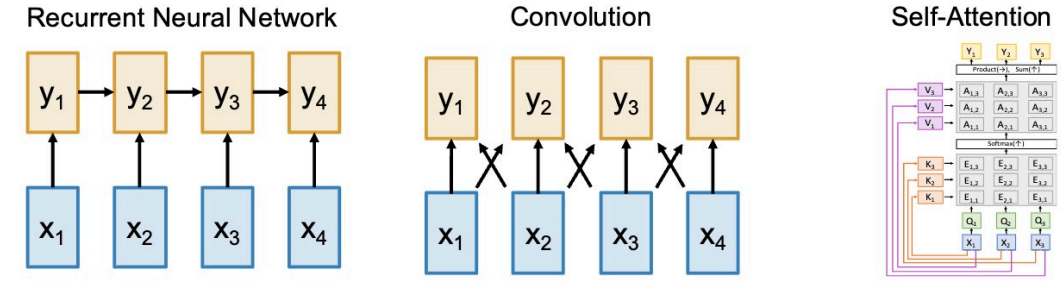
Inputs:
Input vectors: X [N x D_{in}]
Key matrix: W_K [D_{in} x D_{out}]
Value matrix: W_V [D_{in} x D_{out}]
Query matrix: W_Q [D_{in} x D_{out}]

Override similarities with -inf; this controls which inputs each vector is allowed to look at.

Used for language modeling where you want to predict the next word

Computation:
Queries: $Q = XW_Q$ [N x D_{out}]
Keys: $K = XW_K$ [N x D_{out}]
Values: $V = XW_V$ [N x D_{out}]
Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]
$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$
Attention weights: A = softmax(E, dim=1) [N x N]
Output vector: $Y = AV$ [N x D_{out}]
$Y_i = \sum_j A_{ij} V_j$

Three Ways of Processing Sequences



Works on **1D ordered sequences**

(+) Theoretically good at long sequences: O(N) compute and memory for a sequence of length N
(-) Not parallelizable. Need to compute hidden states sequentially

Works on **N-dimensional grids**

(-) Bad for long sequences: need to stack many layers to build up large receptive fields
(+) Parallelizable, outputs can be computed in parallel

Works on **sets of vectors**

(+) Great for long sequences; each output depends directly on all inputs
(+) Highly parallel, it's just 4 matmuls
(-) Expensive: O(N^2) compute, O(N) memory for sequence of length N

Stanford CS231n 10th Anniversary Lecture 8 - 82 April 24, 2025

Transformer

: Self-Attention + MLP + residual + LayerNorm 블록을 쌓은 모델

The Transformer

Transformer Block

Input: Set of vectors x
Output: Set of vectors y

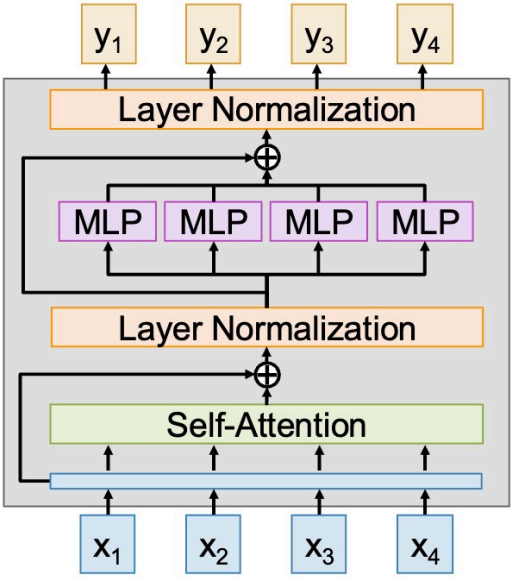
Self-Attention is the only interaction between vectors

LayerNorm and MLP work on each vector independently

Highly scalable and parallelizable, most of the compute is just 6 matmuls:

4 from Self-Attention
2 from MLP

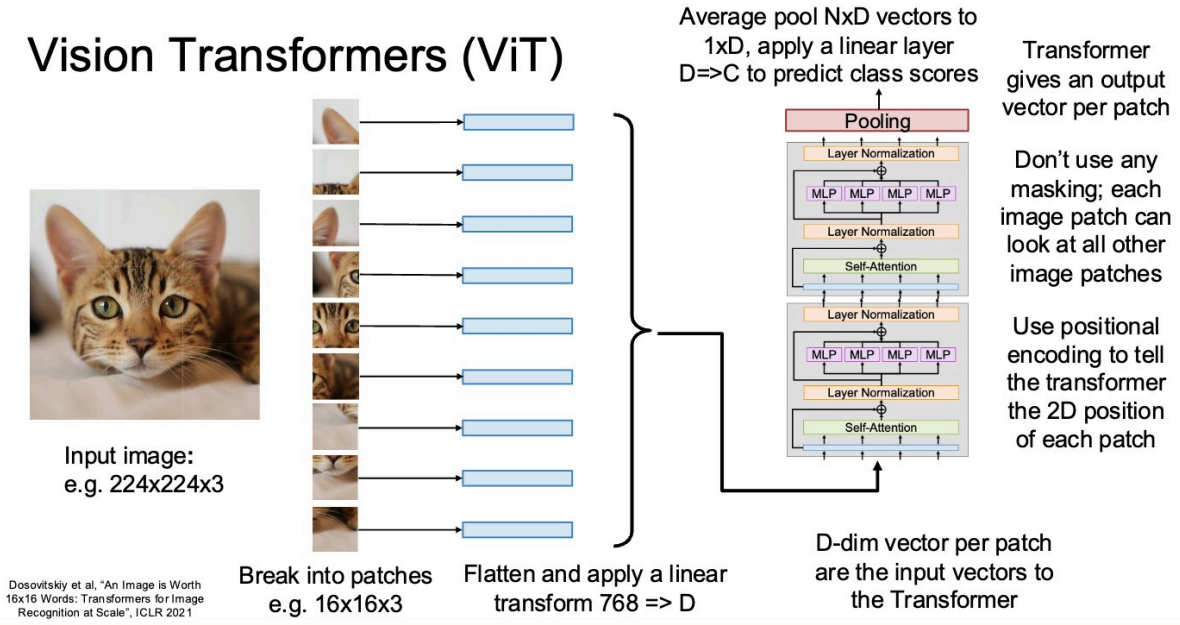
Vaswani et al, "Attention is all you need," NeurIPS 2017



Stanford CS231n 10th Anniversary Lecture 8 - 91 April 24, 2025

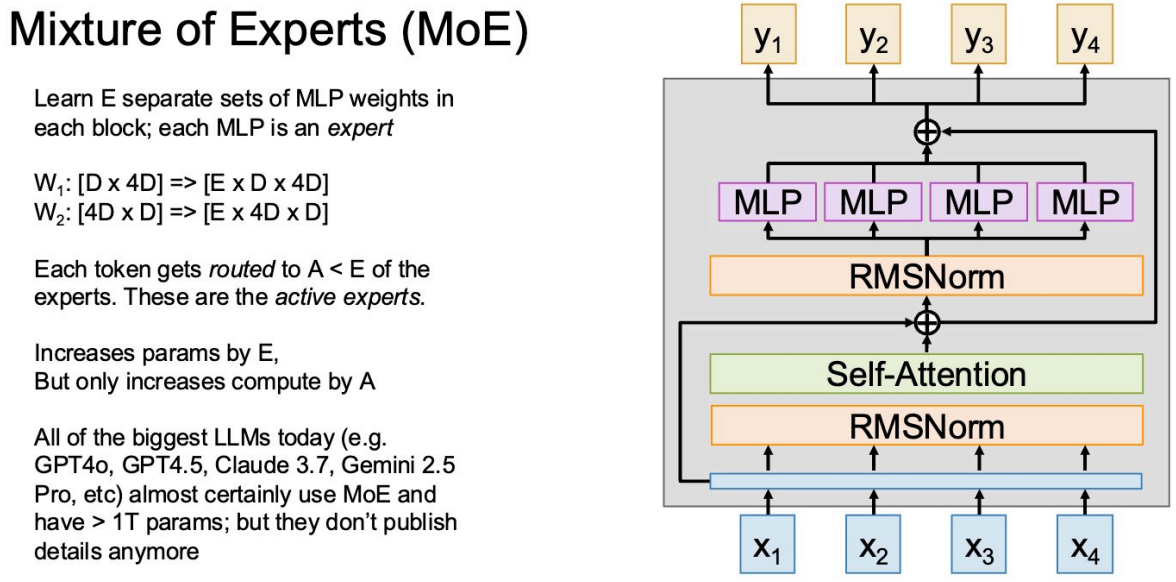
- 토큰끼리 섞이는 곳은 Self-Attention뿐이고 MLP, LayerNorm은 토큰마다 따로 계산
- 행렬 곱 위주라 병렬화가 쉽다 → RNN보다 빠르게 학습

- ViT : 이미지를 16×16 패치로 잘라 토큰처럼 넣는다. 마스크 없이 모든 패치가 서로를 본다.

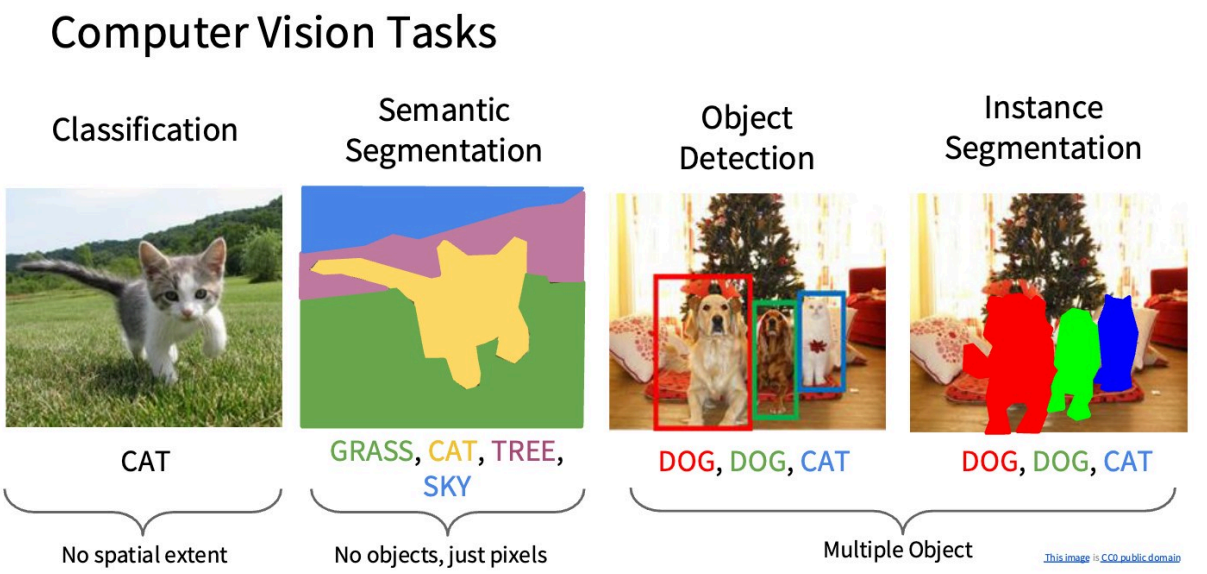


요즘 바뀐 점 :

- Pre-Norm : Norm을 residual 안으로 → 학습 안정
- RMSNorm, SwiGLU : 더 간단한 정규화, 게이트가 있는 MLP
- MoE : 전문가 MLP 여러 개 중 일부만 사용 → 파라미터 ↑, 계산은 조금만 ↑



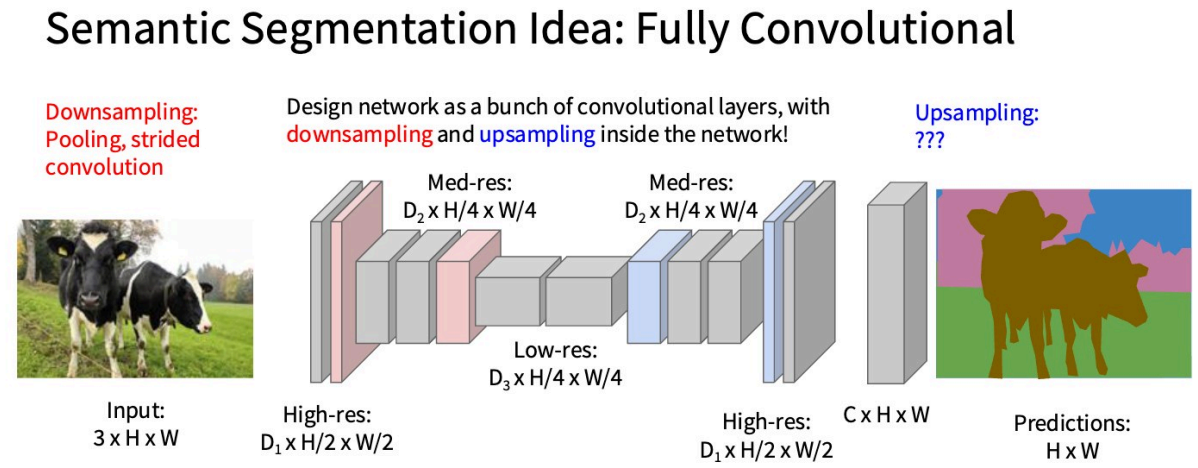
Computer Vision Tasks



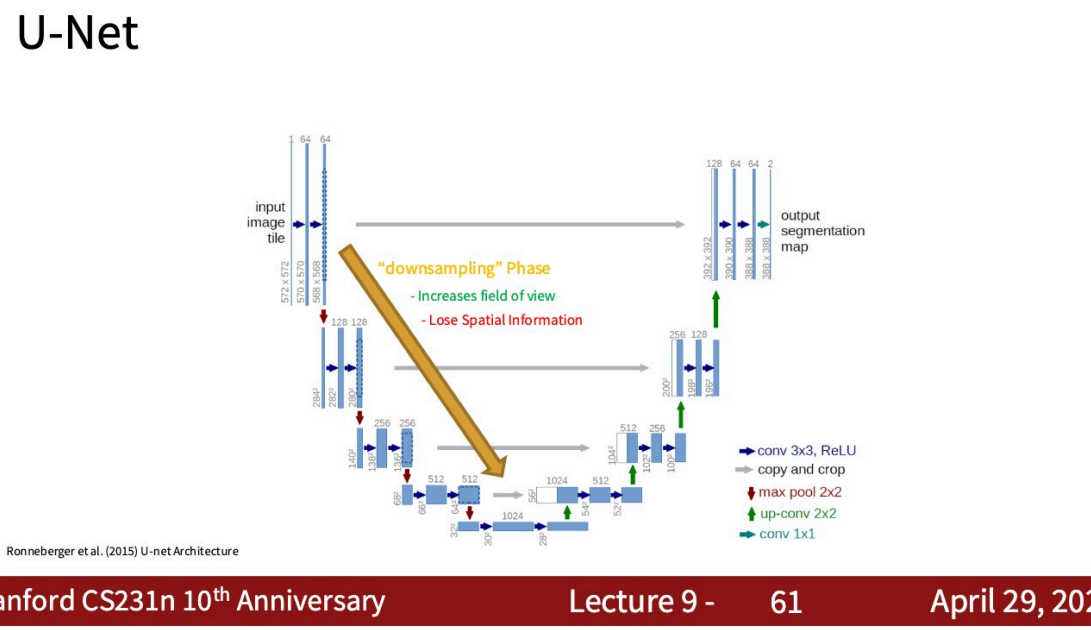
- Semantic Segmentation : 픽셀마다 클래스. 개체 구분 X
- Object Detection : 물체마다 박스 + 클래스
- Instance Segmentation : 물체마다 픽셀 마스크

Semantic Segmentation

: 줄였다가 다시 키운다. 다운샘플링으로 무엇인지 파악하고 업샘플링으로 원래 크기 복원



- 업샘플링 : Unpooling(규칙대로 채우기), Transposed Convolution(학습 가능)
- U-Net : 같은 크기의 앞쪽 특징을 skip 연결로 이어 붙여 경계를 선명하게 복원



Object Detection

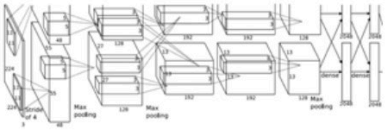
: 물체마다 박스(x, y, w, h)는 회귀로, 클래스는 분류로 예측한다.

Multiple Objects :

Object Detection: Multiple Objects

Each image needs a different number of outputs!

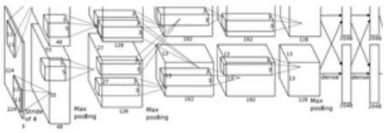




CAT: (x, y, w, h)

4 numbers

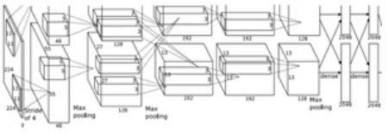




DOG: (x, y, w, h)
DOG: (x, y, w, h)
CAT: (x, y, w, h)

12 numbers





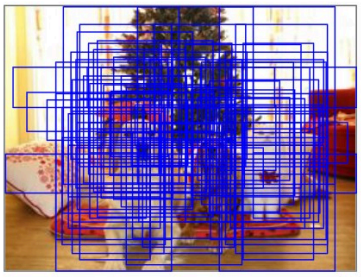
DUCK: (x, y, w, h)
DUCK: (x, y, w, h)
….

Many numbers!

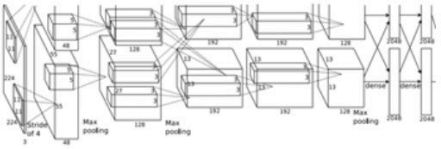
Stanford CS231n 10th Anniversary | Lecture 9 - 72 | April 29, 2025

- 사진마다 물체 수가 달라 출력 개수를 고정할 수 없다. (고양이 1마리 = 숫자 4개, 동물 3마리 = 12개)

Object Detection: Multiple Objects



Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



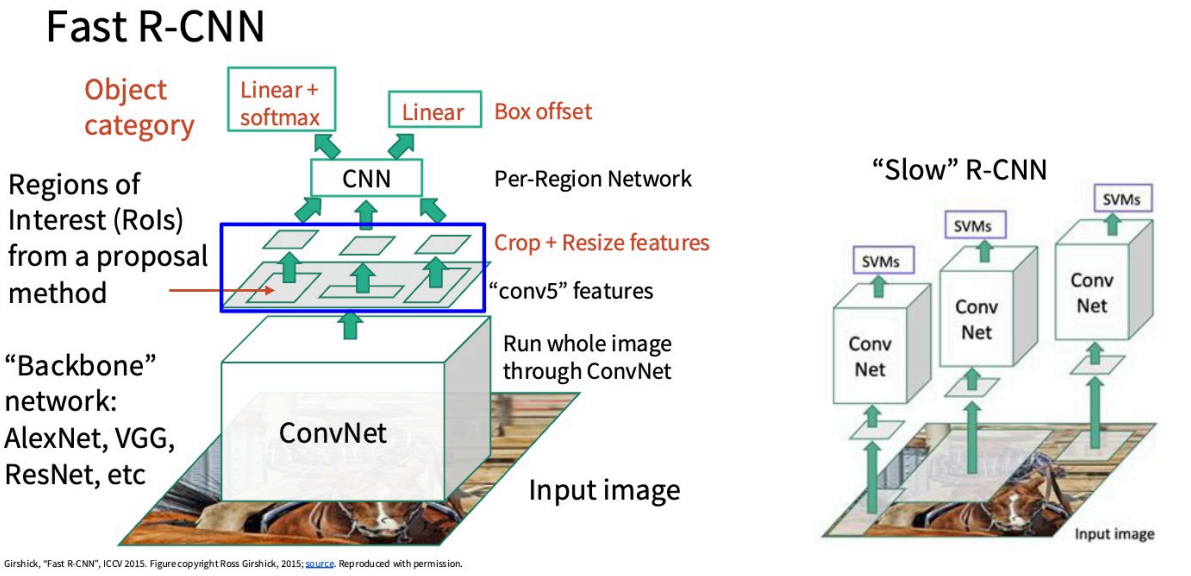
Dog? NO
Cat? YES
Background? NO

Problem: Need to apply CNN to huge number of locations, scales, and aspect ratios, very computationally expensive!

Stanford CS231n 10th Anniversary | Lecture 9 - 77 | April 29, 2025

- 사진을 여러 위치와 크기로 잘라 각각 CNN에 넣고 물체/배경을 분류한다.
- (-) 위치, 크기, 비율 조합이 너무 많아 계산이 감당 안 됨 ⇒ 물체가 있을 법한 후보 영역만 검사하자 (Region Proposal)

모델	방식	한계
R-CNN	후보 영역 약 2000개를 각각 CNN에 넣는다	매우 느림
Fast R-CNN	CNN은 한 번, 특징 맵에서 영역을 잘라 쓴다	후보 영역 찾기가 병목
Faster R-CNN	후보 영역도 RPN(CNN)이 찾는다. two-stage	정확하지만 상대적으로 느림
YOLO / SSD	격자 칸마다 박스와 클래스를 한 번에. single-stage	빠르지만 덜 정확
DETR	Transformer가 박스 집합을 바로 출력	anchor 필요 X



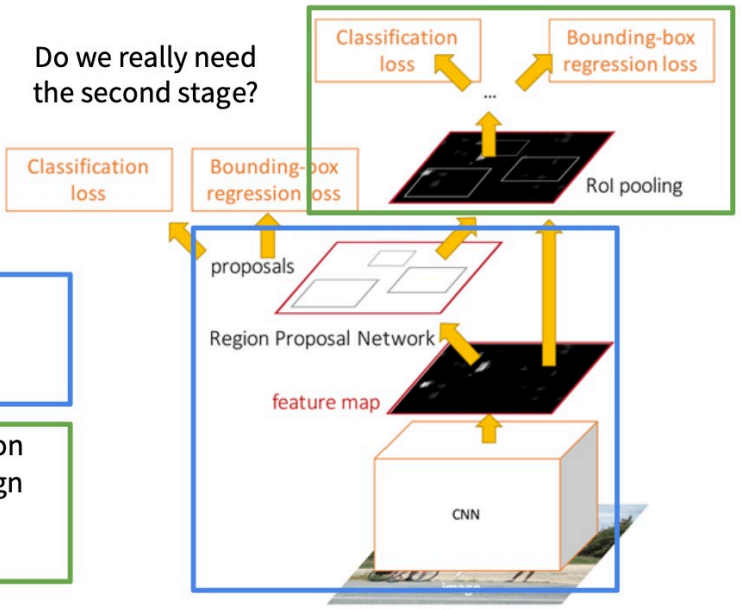
Faster R-CNN:
Make CNN do proposals!

Faster R-CNN is a
Two-stage object detector

First stage: Run once per image
- Backbone network
- Region proposal network

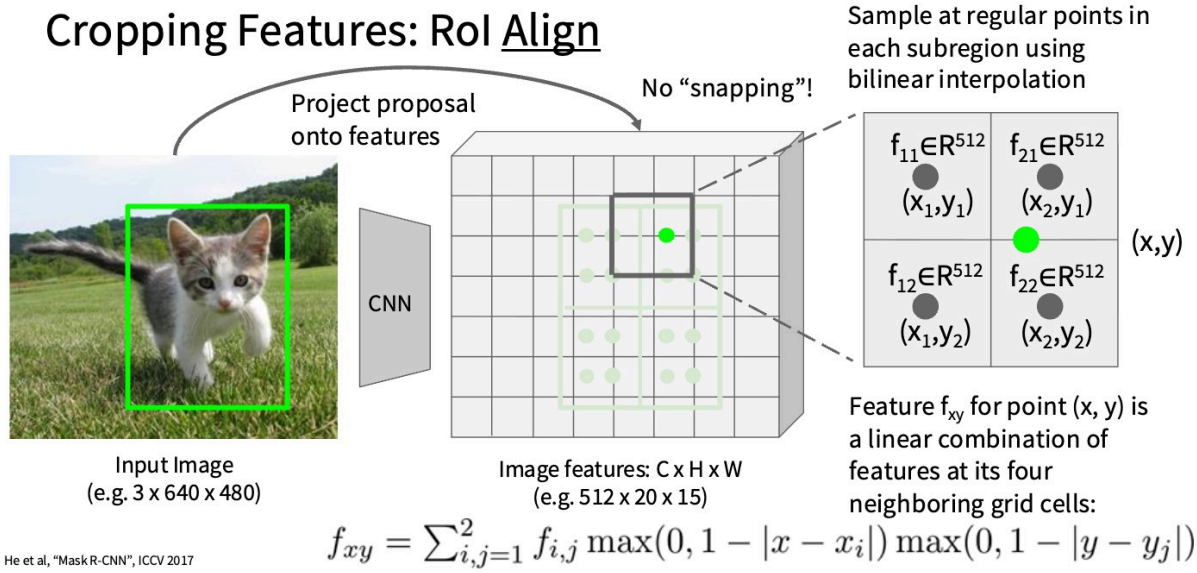
Second stage: Run once per region
- Crop features: RoI pool / align
- Predict object class
- Prediction bbox offset

Do we really need the second stage?



- RoI Pool : 영역을 격자에 맞춰 반올림 → 위치 어긋남 (-)
- RoI Align : 반올림 없이 보간 → 위치 정확

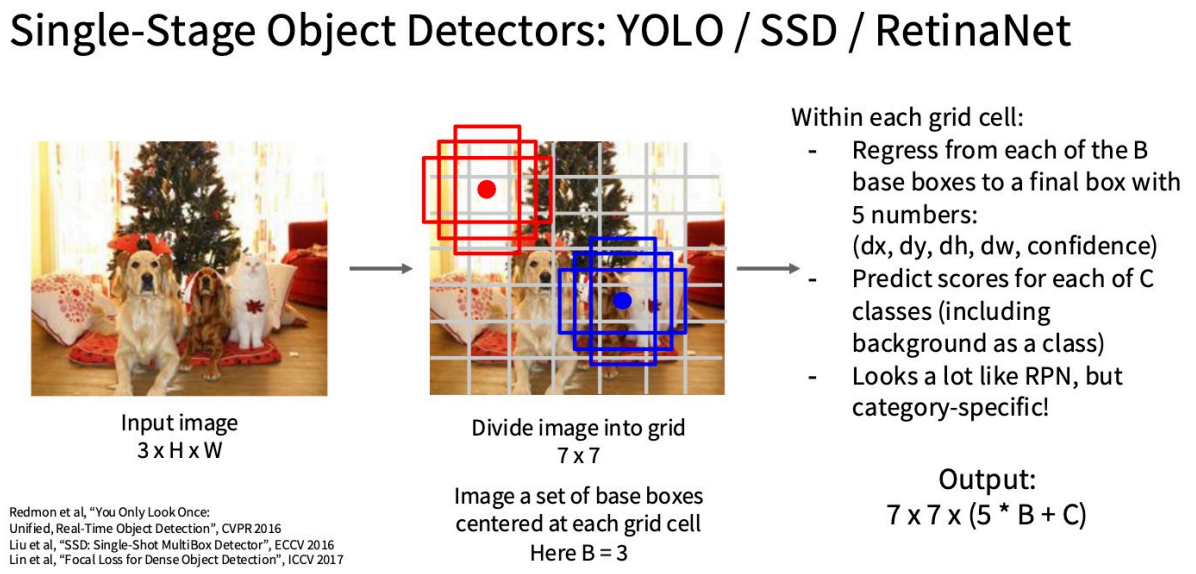
Cropping Features: RoI Align



Feature f_{xy} for point (x, y) is a linear combination of features at its four neighboring grid cells:

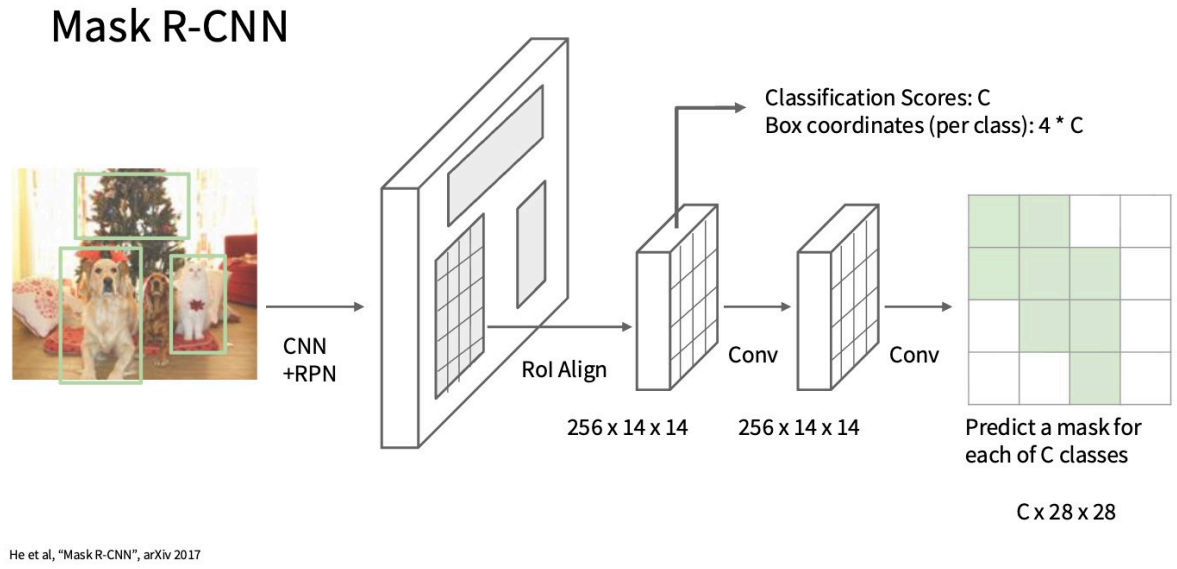
$$f_{xy} = \sum_{i,j=1}^{2} f_{i,j} \max(0, 1 - |x - x_i|) \max(0, 1 - |y - y_j|)$$

He et al, "Mask R-CNN", ICCV 2017



Instance Segmentation

: Mask R-CNN = Faster R-CNN + 영역마다 마스크를 예측하는 작은 네트워크



Visualizing and Understanding

: 모델이 무엇을 보고 판단하는지 들여다본다.

- 첫 레이어 필터 : edge, 색 같은 단순한 패턴

- Saliency Map : 클래스 점수를 입력 픽셀로 미분 → 영향이 큰 픽셀
- CAM : 마지막 특징 맵을 클래스 가중치로 섞은 히트맵. 특정 구조에서만 가능 (-)
- Grad-CAM : 가중치를 기울기 평균으로 구해 어떤 레이어, 구조에도 사용 가능